Tutorial: Conjuntos em Python

Aprenda a criar, modificar, combinar e comparar conjuntos (set) de forma prática.

Objetivo: ao final deste tutorial, você saberá usar os principais recursos de set em Python e reconhecer quando conjuntos são úteis em seus programas.

1. O que são conjuntos?

Em Python, um conjunto (set) é uma coleção de elementos únicos e sem ordem definida. Isso significa que valores repetidos são eliminados automaticamente e que você não deve depender de uma posição, como acontece em listas.

```
frutas = {"maçã", "banana", "laranja"}
print(frutas)
```

> Importante: como conjuntos não são indexados, expressões como frutas[0] não funcionam.

2. Criando um conjunto

Você pode criar um conjunto usando chaves { } ou a função set().

```
numeros = {1, 2, 3, 4}
nomes = {"Ana", "Bruno", "Carlos"}

# A partir de uma lista
lista = [1, 2, 2, 3, 3, 4]
unicos = set(lista)

print(unicos)  # {1, 2, 3, 4}
```

> Para criar um conjunto vazio, use set(). {} cria um dicionário vazio, não um conjunto.

3. Adicionando e removendo elementos

Os métodos mais usados são add(), update(), remove(), discard() e clear().

```
frutas = {"maçã", "banana"}

frutas.add("uva")
frutas.update(["laranja", "manga"])

frutas.remove("banana")   # gera erro se não existir
frutas.discard("kiwi")    # não gera erro se não existir

print(frutas)

frutas.clear()            # remove todos os elementos
```

4. Verificando se um elemento existe

```
cores = {"azul", "verde", "vermelho"}

if "azul" in cores:
    print("Azul está no conjunto!")

if "amarelo" not in cores:
    print("Amarelo não está no conjunto.")
```

O operador in é muito útil para testar rapidamente se um valor pertence ao conjunto.

5. Operações entre conjuntos

Essa é uma das principais vantagens dos conjuntos. Python oferece operações para união, interseção, diferença e diferença simétrica.

Operação	Símbolo	Significado
União	A | B	Elementos de A ou B
Interseção	A & B	Elementos presentes nos dois
Diferença	A - B	Elementos de A que não estão em B
Diferença simétrica	A ^ B	Elementos de A ou B, mas não dos dois

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

print(A | B)  # {1, 2, 3, 4, 5, 6}
print(A & B)  # {3, 4}
print(A - B)  # {1, 2}
print(A ^ B)  # {1, 2, 5, 6}
```

6. Métodos equivalentes

Além dos operadores, existem métodos com nomes mais descritivos:

```
A.union(B)
A.intersection(B)
A.difference(B)
A.symmetric_difference(B)
```

7. Comparando conjuntos

Você pode verificar se um conjunto está contido em outro usando issubset() e issuperset(). Também pode verificar se dois conjuntos não possuem elementos em comum.

```
A = {1, 2}
B = {1, 2, 3, 4}

print(A.issubset(B))   # True
```

```
print(B.issuperset(A)) # True
print(A.isdisjoint({5, 6})) # True
```

8. Percorrendo um conjunto

```
linguagens = {"Python", "Java", "C#"}

for linguagem in linguagens:
    print(linguagem)
```

> A ordem de exibição não deve ser considerada garantida. Se você precisa preservar uma ordem específica, uma lista pode ser mais apropriada.

9. Conjuntos imutáveis: frozenset

O frozenset é uma versão imutável de um conjunto. Depois de criado, seus elementos não podem ser adicionados ou removidos.

```
permissoes = frozenset({"ler", "escrever"})

print(permissoes)

# permissoes.add("excluir")  # Erro
```

10. Quando usar conjuntos?

Conjuntos são especialmente úteis quando você precisa:

- eliminar valores duplicados de uma coleção;
- testar rapidamente se um elemento pertence a um grupo;
- descobrir elementos em comum entre grupos;
- comparar listas de valores únicos;
- realizar operações matemáticas de conjuntos.

11. Exemplo prático

Imagine duas turmas e você quer descobrir quais alunos estão nas duas:

```
turma_a = {"Ana", "Bruno", "Carlos", "Diana"}
turma_b = {"Carlos", "Diana", "Eduardo", "Fernanda"}

em_ambas = turma_a & turma_b
somente_a = turma_a - turma_b
todos = turma_a | turma_b

print("Nas duas turmas:", em_ambas)
print("Somente na turma A:", somente_a)
print("Todos os alunos:", todos)
```

12. Exercícios

Tente resolver sem consultar a resposta primeiro.

Exercício 1: crie um conjunto com os números 1 a 5 e adicione o número 6.

Exercício 2: transforme a lista [1, 2, 2, 3, 4, 4, 5] em um conjunto.

Exercício 3: dados A = {1, 2, 3} e B = {3, 4, 5}, encontre a união e a interseção.

Exercício 4: descubra quais nomes aparecem em duas listas usando conjuntos.

Gabarito

```
# 1
numeros = {1, 2, 3, 4, 5}
numeros.add(6)

# 2
dados = [1, 2, 2, 3, 4, 4, 5]
unicos = set(dados)

# 3
A = {1, 2, 3}
B = {3, 4, 5}
uniao = A | B
intersecao = A & B

# 4
lista1 = ["Ana", "Bruno", "Carlos"]
lista2 = ["Carlos", "Diana", "Ana"]
comuns = set(lista1) & set(lista2)
```

Resumo rápido

set() cria um conjunto; add() adiciona um elemento; remove()/discard() removem; | faz união; & faz interseção; - faz diferença; ^ faz diferença simétrica; in verifica pertencimento; frozenset() cria um conjunto imutável.